

1 | Contest Problems Upsolving

A. Lower Bound-STL

Problem Statement

You are given a sorted array of integers and multiple queries. For each query, you must check whether the number exists in the array. If it exists, print "Yes" with its first 1-based index. If it does not exist, print "No" along with the index of the smallest number greater than it. Since the array is sorted, this is efficiently solved using lower bound (binary search)

Theory / Approach

The solution uses lower bound (binary search) to find the first position where the given number can be inserted while maintaining sorted order. If the element at that position matches the query value, it is present in the array. Otherwise, that position gives the index of the next greater element. This allows each query to be answered efficiently in logarithmic time

Code

```
Contest > G++ a.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8
9      int arr[n];
10     for (int i = 0; i < n; i++)
11     {
12         cin >> arr[i];
13     }
14
15     int q;
16     cin >> q;
17
18     while (q)
19     {
20         int x;
21         cin >> x;
22
23         int *it = lower_bound(arr, arr + n, x);
24         int idx = it - arr + 1;
25
26         if (it != arr + n && *it == x)
27         {
28             cout << "Yes " << idx << endl;
29         }
30
31         else
32         {
33             cout << "No " << idx << endl;
34         }
35         q--;
36     }
37
38     return 0;
39 }
```

Output

```
8
1 1 2 2 6 9 9 15
4
1
Yes 1
4
No 5
9
Yes 6
15
Yes 8
```

Verdict

68990779	fardinnuman	A	Accepted	1274	544	C++	14 days ago
----------	-------------	---	----------	------	-----	-----	-------------

Discussion

- **Time Complexity:** $O((n + q)\log n)$, as each query uses binary search
- **Space Complexity:** $O(n)$, for storing the input array

B. Subarray Division

Problem Statement

You are given an array representing chocolate squares and two values: a target sum (birth day) and a segment length (birth month). You need to count how many contiguous segments of size equal to the birth month have a sum equal to the birth day

Theory / Approach

The solution uses a sliding window technique. First, compute the sum of the first m elements. Then slide the window one step at a time by removing the leftmost element and adding the next element. At each step, check if the current window sum equals the target d . This avoids recomputing sums from scratch and ensures an efficient linear scan through the array

Code

```
Context > G++ b.cpp > _
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8
9      int arr[n];
10     for (int i = 0; i < n; i++)
11     {
12         cin >> arr[i];
13     }
14
15     int day, month;
16     cin >> day >> month;
17
18     int count = 0, sum = 0;
19
20     for (int i = 0; i < month && i < n; i++)
21     {
22         sum = sum + arr[i];
23     }
24
25     if (sum == day)
26         count++;
27
28     for (int i = month; i < n; i++)
29     {
30         sum += arr[i] - arr[i - month];
31         if (sum == day)
32             count++;
33     }
34
35     cout << count << endl;
36
37     return 0;
38 }
```

Output

```
5
1 2 1 3 2
3 2
2
```

Verdict

68991268 fardinnuman B Accepted 128 546 C++ 14 days ago

Discussion

- **Time Complexity:** $O(n)$, since each element is processed once using sliding window
- **Space Complexity:** $O(1)$, as only a few variables are used for computation

C. Sherlock and Anagrams

Problem Statement

You are given a string and need to find how many pairs of substrings are anagrams of each other. Two substrings form a valid pair if one can be rearranged to form the other. The task is to count all such unordered anagrammatic pairs in the given string

Theory / Approach

The solution generates all possible substrings and represents each substring using a frequency signature of 26 characters (count of each letter). Substrings with the same frequency signature are anagrams. We store each signature in a map and count occurrences. Finally, for each group of identical signatures, we compute pairs using the formula $(c \times (c - 1) / 2)$, where (c) is the frequency of that signature

Code

```
Contest > G++ ccpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int t;
7      cin >> t;
8
9      while (t--)
10     {
11         string s;
12         cin >> s;
13         int n = s.size();
14
15         map<string, int> mp;
16
17         for (int i = 0; i < n; i++)
18         {
19             vector<int> freq(26, 0);
20
21             for (int j = i; j < n; j++)
22             {
23                 freq[s[j] - 'a']++;
24
25                 string key = "";
26                 for (int k = 0; k < 26; k++)
27                 {
28                     key += char(freq[k] + '0');
29                 }
30
31                 mp[key]++;
32             }
33         }
34
35         int ans = 0;
36         for (auto x : mp)
37         {
38             int c = x.second;
39             ans += c * (c - 1) / 2;
40         }
41
42         cout << ans << endl;
43     }
44
45     return 0;
46 }
```

Output

```
2
abba
4
abcd
0
```

Verdict

69303832	fardinnuman	C	Accepted	118	777	C++	4 hr ago
----------	-------------	---	----------	-----	-----	-----	----------

Discussion

- **Time Complexity:** $O(n^2)$, due to generating all substrings and building frequency keys
- **Space Complexity:** $O(n^2)$, for storing substring signatures in the map

D. Task Scheduling

Problem Statement

You are given a list of tasks where each task has a processing time and a deadline. Tasks can be split and resumed later. You must schedule all tasks in a way that minimizes the maximum delay (lateness), where lateness is the amount by which completion time exceeds the deadline. For each prefix of tasks, you must **Output** the minimum possible maximum lateness

Theory / Approach

The optimal strategy for minimizing maximum lateness is to process tasks in ascending order of deadlines (Earliest Deadline First idea). However, because tasks can be split and resumed, the problem effectively reduces to tracking how completion times accumulate relative to deadlines. For each prefix, we simulate scheduling under optimal ordering and compute the maximum overshoot among tasks

Discussion

This problem was not successfully solved during the lab contest due to its high conceptual complexity and non-trivial scheduling behavior. The main difficulty was understanding how task splitting affects completion time and how to correctly model an optimal schedule. Another challenge was determining how to efficiently compute the maximum lateness for each prefix without fully simulating all possible interleavings. The combination of greedy scheduling intuition and prefix-wise evaluation made it difficult to implement correctly within the time limit

2 | Prime Factorization (2 Problems)

1. T-primes

Problem Statement

We are given multiple integers and need to determine for each whether it is a T-prime, meaning it has exactly three distinct positive divisors. For every number, we print "YES" if it is T-prime, otherwise "NO"

Theory / Approach

A number has exactly three divisors only when it is a perfect square of a prime number. So instead of directly counting divisors, we check two conditions: (1) the number must be a perfect square, and (2) its square root must be a prime number. We first precompute all primes up to (10^6) using the Sieve of Eratosthenes. Then, for each query number, we compute its square root and verify both conditions efficiently

Code

```
G++ 1.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int N = 1000000;
5  bool prime[N + 1];
6
7  void sieve()
8  {
9      fill(prime, prime + N + 1, true);
10     prime[0] = prime[1] = false;
11     for (int i = 2; i * i <= N; i++)
12     {
13         if (prime[i])
14         {
15             for (int j = i * i; j <= N; j += i)
16                 prime[j] = false;
17         }
18     }
19 }
20
21 int main()
22 {
23     ios::sync_with_stdio(false);
24     cin.tie(nullptr);
25
26     sieve();
27
28     int n;
29     cin >> n;
30
31     while (n--)
32     {
33         long long x;
34         cin >> x;
35
36         long long r = sqrt(x);
37
38         if (r * r == x && prime[r])
39             cout << "YES\n";
40         else
41             cout << "NO\n";
42     }
43
44     return 0;
45 }
```

Output

```
3
4 5 6
YES
NO
NO
```

Verdict

371589696 Apr/18/2026 09:51 UTC+6 fardinnuman 230B - T-primes C++17 (GCC 7-32) Accepted 156 ms 1000 KB

Discussion

- **Time Complexity:** $O(n + N)$
- **Space Complexity:** $O(N)$ for prime sieve array

2. Game 23

Problem Statement

We are given two integers n and m . We can transform n into m only by multiplying it by 2 or 3 any number of times. We need to find the minimum number of operations required, or print -1 if it is not possible

Theory / Approach

To check feasibility, we first verify whether m is divisible by n . If not, transformation is impossible. Then we reduce the ratio m / n step by step by dividing out all factors of 2 and 3. Each division counts as one operation. If after removing all 2s and 3s the result becomes 1, transformation is valid; otherwise, it is impossible. The total number of divisions gives the answer

Code

```
G++ 2.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      long long n, m;
7      cin >> n >> m;
8
9      if (m % n != 0)
10     {
11         cout << -1;
12         return 0;
13     }
14
15     long long x = m / n;
16     int moves = 0;
17
18     while (x % 2 == 0)
19     {
20         x /= 2;
21         moves++;
22     }
23
24     while (x % 3 == 0)
25     {
26         x /= 3;
27         moves++;
28     }
29
30     if (x != 1)
31         cout << -1;
32     else
33         cout << moves;
34
35     return 0;
36 }
```

Output

```
120 51840
7
```

Verdict

371589770	Apr/18/2026 09:52UTC+6	fardinnuman	1141A - Game 23	C++17 (GCC 7-32)	Accepted	31 ms	100 KB
-----------	------------------------	-------------	-----------------	------------------	----------	-------	--------

Discussion

- **Time Complexity:** $O(\log n)$
- **Space Complexity:** $O(1)$

3 | Euclidean Practice (1 Problem)

1. Maximum Sum of Digits

Problem Statement

We are given an integer n and must split it into two non-negative integers a and b such that $a + b = n$. The goal is to maximize the sum of digits of a and b combined, i.e., $S(a) + S(b)$

Theory / Approach

Instead of trying all splits, we use a greedy idea: the optimal split occurs when we distribute digits in a way that avoids carrying during addition. A key observation is that maximizing digit sums is achieved when we break n into two numbers where digits are balanced greedily from right to left. Practically, we iterate through digits and construct a and b so their digit-wise sum never exceeds 9, maximizing total digit contribution without carry loss

Code

```
G++1.cpp
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int sumDigits(long long x)
5 {
6     int s = 0;
7     while (x)
8     {
9         s += x % 10;
10        x /= 10;
11    }
12    return s;
13 }
14
15 int main()
16 {
17     long long n;
18     cin >> n;
19
20     int ans = sumDigits(n);
21
22     long long t = 0;
23     long long p = 1;
24
25     while (p <= n)
26     {
27         t = t + 9 * p;
28         if (t <= n)
29         {
30             long long a = t;
31             long long b = n - a;
32             ans = max(ans, sumDigits(a) + sumDigits(b));
33         }
34         p *= 10;
35     }
36
37     cout << ans;
38     return 0;
39 }
```

Output

```
35
17
```

Verdict

371590428	Apr/18/2026 10:04UTC+6	fardinnuman	1060B - Maximum Sum of Digits	C++17 (GCC 7-32)	Accepted	62 ms	100 KB
-----------	------------------------	-------------	-------------------------------	------------------	----------	-------	--------

Discussion

- **Time Complexity:** $O(d)$, where d is number of digits in n
- **Space Complexity:** $O(d)$ for digit storage

4 | Sieve (3 Problem)

1. Sherlock and his girlfriend

Problem Statement

Given n jewelry pieces with prices $2, 3, 4, \dots, n+1$. Color them so that if price x is a prime divisor of price y , they have different colors. Minimize the number of colors used

Theory / Approach

- If $n \leq 2$, only 1 color is needed because no prime divides another prime.
- If $n \geq 3$, 2 colors suffice:
- Color 1 for all prime numbers
- Color 2 for all composite numbers
- A composite number always has a prime divisor, so it will always have a different color than its prime factor.

Code

```
G++ App > ...
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main()
6  {
7      int n;
8      cin >> n;
9
10     if (n <= 2)
11     {
12         cout << 1 << endl;
13         for (int i = 0; i < n; i++)
14         {
15             cout << 1 << " ";
16         }
17         cout << endl;
18     }
19     else
20     {
21         cout << 2 << endl;
22         for (int i = 2; i <= n + 1; i++)
23         {
24             bool isPrime = true;
25             for (int j = 2; j * j <= i; j++)
26             {
27                 if (i % j == 0)
28                 {
29                     isPrime = false;
30                     break;
31                 }
32             }
33
34             if (isPrime)
35             {
36                 cout << 1 << " ";
37             }
38             else
39             {
40                 cout << 2 << " ";
41             }
42         }
43         cout << endl;
44     }
45     return 0;
46 }
47
48 }
```

Output

```
3
2
1 1 2
```

Verdict

371590884	Apr/18/2026 10:13 UTC+6	fardinnuman	B - Sherlock and his girlfriend	C++17 (GCC 7-32)	Accepted	78 ms	0 KB
---------------------------	----------------------------	-----------------------------	---	------------------	----------	-------	------

Discussion

- Time Complexity: $O(n\sqrt{n})$, primality check for each number up to $n+1$
- Space Complexity: $O(1)$

2. Next Test

Problem Statement

Given n distinct positive integers (test indices), find the smallest positive integer that is not present in the list.

Theory / Approach

- Sort the array of existing test indices
- Iterate from 1 upwards, checking if each number exists in the sorted array
- The first missing number is the answer
- Since $n \leq 3000$ and $a_i \leq 3000$, the answer will be at most 3001

Code

```
G++ 5.cpp > ...
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  int main()
6  {
7      int n;
8      cin >> n;
9
10     int a[3005];
11     for (int i = 0; i < n; i++)
12     {
13         cin >> a[i];
14     }
15
16     sort(a, a + n);
17
18     int ans = 1;
19     for (int i = 0; i < n; i++)
20     {
21         if (a[i] == ans)
22         {
23             ans++;
24         }
25         else
26         {
27             break;
28         }
29     }
30
31     cout << ans << endl;
32     return 0;
33 }
34
```

Output

```
3
1 7 2
3
```

Verdict

371591044	Apr/18/2026 10:17 UTC+6	fardinnuman	A - Next Test	C++17 (GCC 7-32)	Accepted	92 ms	100 KB
---------------------------	----------------------------	-------------	-------------------------------	------------------	----------	-------	--------

Discussion

- Time Complexity: $O(n \log n)$, due to sorting
- Space Complexity: $O(n)$

3. Olesya and Rodion

Problem Statement

Given n (number of digits) and t (divisor, $2 \leq t \leq 10$), find an n -digit positive integer (no leading zeros) divisible by t . If none exists, print -1

Theory / Approach

- If $t = 10$, the number must end with 0. For $n = 1$, an n -digit number ending with 0 is just "0" which is not positive. So for $n = 1$ and $t = 10$, answer is -1
- For $t = 10$ and $n \geq 2$, print '1' followed by $(n-1)$ zeros
- For all other t (2-9), construct an n -digit number by writing t repeated n times. This number will be divisible by t
- Special case: If $t = 10$ and $n = 1 \rightarrow -1$

Code

```
G++6.cpp >...
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n, t;
7      cin >> n >> t;
8
9      if (t == 10)
10     {
11         if (n == 1)
12         {
13             cout << -1 << endl;
14         }
15         else
16         {
17             cout << 1;
18             for (int i = 1; i < n; i++)
19             {
20                 cout << 0;
21             }
22             cout << endl;
23         }
24     }
25     else
26     {
27         for (int i = 0; i < n; i++)
28         {
29             cout << t;
30         }
31         cout << endl;
32     }
33
34     return 0;
35 }
```

Output

```
3 2
222
```

Verdict

371591086	Apr/18/2026 10:18 UTC+6	fardinnuman	A - Olesya and Rodion	C++17 (GCC 7-32)	Accepted	46 ms	100 KB
---------------------------	----------------------------	-----------------------------	---------------------------------------	------------------	----------	-------	--------

Discussion

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

4 | Cumulative Sum (2 Problems)

1. Kuriyama Mirai's Stones

Problem Statement

Given n stones with costs $v[i]$. Two types of queries:

- Type 1: sum of costs from index l to r in the original order
- Type 2: sum of costs from index l to r in sorted (non-decreasing) order

Answer m queries efficiently

Theory / Approach

- Use prefix sum arrays for $O(1)$ range sum queries
- Create prefix sum array `prefix\_original` where $\text{prefix_original}[i] = \text{sum of } v[1..i]$
- Sort v to get u (non-decreasing order)
- Create prefix sum array `prefix\_sorted` where $\text{prefix_sorted}[i] = \text{sum of } u[1..i]$
- For each query:
 - Type 1: Output $\text{prefix_original}[r] - \text{prefix_original}[l-1]$
 - Type 2: Output $\text{prefix_sorted}[r] - \text{prefix_sorted}[l-1]$
- Use long long to avoid overflow (v_i up to $1e9$, n up to $1e5 \rightarrow \text{sum up to } 1e14$)

Code

```
G++ 7.cpp > ...
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  int main()
6  {
7      ios_base::sync_with_stdio(false);
8      cin.tie(NULL);
9
10     int n;
11     cin >> n;
12
13     long long v[100005];
14     long long prefix_original[100005] = {0};
15
16     for (int i = 1; i <= n; i++)
17     {
18         cin >> v[i];
19         prefix_original[i] = prefix_original[i - 1] + v[i];
20     }
21
22     sort(v + 1, v + n + 1);
23
24     long long prefix_sorted[100005] = {0};
25     for (int i = 1; i <= n; i++)
26     {
27         prefix_sorted[i] = prefix_sorted[i - 1] + v[i];
28     }
29
30     int m;
31     cin >> m;
32
33     while (m--)
34     {
35         int type, l, r;
36         cin >> type >> l >> r;
37
38         if (type == 1)
39         {
40             cout << prefix_original[r] - prefix_original[l - 1] << "\n";
41         }
42         else
43         {
44             cout << prefix_sorted[r] - prefix_sorted[l - 1] << "\n";
45         }
46     }
47
48     return 0;
49 }
```

Output

```
6
6 4 2 7 2 7
3
2 3 6
1 3 4
1 1 6
24
9
28
```

Verdict

371591269	Apr/18/2026 10:22 UTC+6	fardinnuman	B - Kuriyama Mirai's Stones	C++17 (GCC 7-32)	Accepted	140 ms	100 KB
---------------------------	----------------------------	-------------	---	------------------	----------	--------	--------

Discussion

- Time Complexity: $O(n \log n + m)$, sorting takes $O(n \log n)$, queries take $O(1)$ each
- Space Complexity: $O(n)$, for storing prefix sums

2. Fence

Problem Statement

Given n planks with heights $h[i]$, find the starting index j of k consecutive planks (j to $j+k-1$) with the minimum sum of heights. If multiple answers exist, print any

Theory / Approach

- Use sliding window technique to calculate sum of each window of size k in $O(n)$
- Initialize sum of first k planks (indices 0 to $k-1$)
- Slide window by subtracting leftmost element and adding next element
- Track minimum sum and its starting index (1-based Output)
- Since $h_i \leq 100$, sums fit in `int`

Code

```
G++ 8.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n, k;
7      cin >> n >> k;
8
9      int h[150005];
10     for (int i = 0; i < n; i++)
11     {
12         cin >> h[i];
13     }
14
15     int current_sum = 0;
16     for (int i = 0; i < k; i++)
17     {
18         current_sum += h[i];
19     }
20
21     int min_sum = current_sum;
22     int min_index = 1;
23
24     for (int i = k; i < n; i++)
25     {
26         current_sum = current_sum - h[i - k] + h[i];
27
28         if (current_sum < min_sum)
29         {
30             min_sum = current_sum;
31             min_index = i - k + 2;
32         }
33     }
34
35     cout << min_index << endl;
36
37     return 0;
38 }
```

Output

```
7 3
1 2 6 1 1 7 1
3
```

Verdict

371591372	Apr/18/2026 10:24 UTC+6	fardinuman	B - Fence	C++17 (GCC 7-32)	Accepted	171 ms	100 KB
---------------------------	----------------------------	----------------------------	---------------------------	------------------	----------	--------	--------

Discussion

- Time Complexity: $O(n)$, single pass through the array
- Space Complexity: $O(n)$, for storing heights (can be $O(1)$ by processing on the fly)

5 | Greedy (3 Problems)

1. Kefa and First Steps

Problem Statement

Given an array a of length n , find the length of the longest contiguous non-decreasing subsegment (i.e., each element is $\geq$ the previous one)

Theory / Approach

- Traverse the array once, keeping track of current non-decreasing segment length
- Start with `current_length = 1` (first element)
- For i from 1 to $n-1$:
- If $a[i] \geq a[i-1]$, increment `current_length`
- Else, reset `current_length = 1`
- Update `max_length = max(max_length, current_length)`
- Output `max_length`

Code

```
G++ 9.cpp > ...
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  int main()
6  {
7      int n;
8      cin >> n;
9
10     int a[100005];
11     for (int i = 0; i < n; i++)
12     {
13         cin >> a[i];
14     }
15
16     int max_len = 1;
17     int current_len = 1;
18
19     for (int i = 1; i < n; i++)
20     {
21         if (a[i] >= a[i - 1])
22         {
23             current_len++;
24             max_len = max(max_len, current_len);
25         }
26         else
27         {
28             current_len = 1;
29         }
30     }
31
32     cout << max_len << endl;
33
34     return 0;
35 }
```

Output

```
6
2 2 1 3 4 1
3
```

Verdict

371591508	Apr/18/2026 10:26 UTC+6	fardinnuman	A - Kefa and First Steps	C++17 (GCC 7-32)	Accepted	187 ms	100 KB
---------------------------	----------------------------	-----------------------------	--	------------------	----------	--------	--------

Discussion

- Time Complexity: $O(n)$, single pass through the array
- Space Complexity: $O(n)$, for storing array (can be $O(1)$ by processing on the fly)

2. Puzzles

Problem Statement

Given n students and m puzzles with piece counts $f[i]$, choose exactly n puzzles such that the difference between the maximum and minimum piece count among chosen puzzles is minimized.

Output that minimum difference

Theory / Approach

- Sort the array of puzzle piece counts
- Since we need n consecutive puzzles in sorted order to minimize max-min difference (any gap would only increase difference)
- Use sliding window of size n over the sorted array
- For each window, compute difference = $f[i+n-1] - f[i]$
- Track the minimum difference across all windows
- Return the minimum

Code

```
G++ 10.cpp > ...
1  #include <iostream>
2  #include <algorithm>
3  #include <climits>
4  using namespace std;
5
6  int main()
7  {
8      int n, m;
9      cin >> n >> m;
10
11     int f[55];
12     for (int i = 0; i < m; i++)
13     {
14         cin >> f[i];
15     }
16
17     sort(f, f + m);
18
19     int min_diff = INT_MAX;
20
21     for (int i = 0; i <= m - n; i++)
22     {
23         int diff = f[i + n - 1] - f[i];
24         if (diff < min_diff)
25         {
26             min_diff = diff;
27         }
28     }
29
30     cout << min_diff << endl;
31
32     return 0;
33 }
```

Output

```
4 6
10 12 10 7 5 22
5
```

Verdict

371591563	Apr/18/2026 10:27 UTC+6	fardinnuman	A - Puzzles	C++17 (GCC 7-32)	Accepted	92 ms	0 KB
---------------------------	----------------------------	-------------	-----------------------------	------------------	----------	-------	------

Discussion

- Time Complexity: $O(m \log m)$, due to sorting
- Space Complexity: $O(m)$, for storing array

3. Boredom

Problem Statement

Given a sequence of n integers, in one move you can pick a number a_k , delete all occurrences of a_k , a_k+1 , and a_k-1 from the sequence, and gain a_k points. Find the maximum total points possible

Theory / Approach

- This is a dynamic programming problem similar to House Robber
- First, count frequency of each value (up to 10^5)
- Let $dp[i]$ = maximum points achievable considering numbers from 1 to i
- Transition:
 - If we take value i : $points = freq[i] * i + dp[i-2]$ (since $i-1$ becomes unavailable)
 - If we skip value i : $points = dp[i-1]$
- $dp[i] = \max(dp[i-1], dp[i-2] + freq[i] * i)$
- Base cases:
 - $dp[0] = 0$
 - $dp[1] = freq[1] * 1$
- Answer is $dp[max_value]$

Code

```
G++ 11.cpp > ...
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  int main()
6  {
7      int n;
8      cin >> n;
9
10     long long freq[100005] = {0};
11     int max_val = 0;
12
13     for (int i = 0; i < n; i++)
14     {
15         int x;
16         cin >> x;
17         freq[x]++;
18         max_val = max(max_val, x);
19     }
20
21     long long dp[100005] = {0};
22     dp[1] = freq[1] * 1;
23
24     for (int i = 2; i <= max_val; i++)
25     {
26         dp[i] = max(dp[i-1], dp[i-2] + freq[i] * i);
27     }
28
29     cout << dp[max_val] << endl;
30
31     return 0;
32 }
```

Output

```
2
1 2
2
```

Verdict

371591601	Apr/18/2026 10:28 ^{UTC+6}	fardinnuman	A - Boredom	C++17 (GCC 7-32)	Accepted	140 ms	100 KB
---------------------------	---------------------------------------	-------------	-----------------------------	------------------	----------	--------	--------

Discussion

- Time Complexity: $O(n + \text{max_val})$, where $\text{max_val} \leq 10^5$
- Space Complexity: $O(\text{max_val})$, for frequency and dp arrays